Flutter Assignment: Smart Assistant App

Objective

- Create clean UI with Flutter
- Integrate APIs and manage state
- Apply Flutter core concepts effectively
- Handle navigation, theming, and responsive layouts

You will consume a dummy API to build a simple Smart Assistant experience.

Problem Statement

- View a list of suggestions using pagination
- Chat with a dummy assistant
- View chat history

The app should simulate how a real-world mobile assistant app works.

Dummy API Details

1. Get Suggestions (Paginated)

```
GET /suggestions?page={page}&limit={limit}
```

Query Parameters:

- current_page (integer, default: 1) — Page number to retrieve
- limit (integer, default: 10) — Number of suggestions per page

Response:

```
{
 "status": "success",
 "data": [
  {
   "id": 1,
   "title": "Summarize my notes",
   "description": "Get a concise summary of your text"
  },
  {
```

```
    "id": 2,
    "title": "Generate email reply",
    "description": "Create a professional email response"
  },
 ],
 "pagination": {
  "current_page": 1,
  "total_pages": 5,
  "total_items": 50,
  "limit": 10,
  "has_next": true,
  "has_previous": false
 }
}
```

2. Send Chat Message

```
POST /chat
```

Request Body:

```
{ "message": "Explain Flutter state management" }
```

Response:

```
{
    "status": "success",
    "reply": "Flutter state management helps you manage UI updates efficiently..." }
```

3. Get Chat History

```
GET /chat/history
```

Response:

```
{
 "status": "success",
 "data": [
  { "sender": "user", "message": "What is Flutter?" },
  { "sender": "assistant", "message": "Flutter is an open-source UI toolkit by Google..." }
 ]
}
```

App Requirements

Screens

- Home Screen – Suggestions with pagination
- Chat Screen – Chat UI, input field, loading indicator
- History Screen – Previous chat messages

Technical Requirements

- Flutter (latest stable)
- Clean folder structure
- State management: Provider / Riverpod / Bloc
- Pagination implementation
- Correct widget lifecycle management

Evaluation Criteria

Area	What We Look For
Flutter Basics	Widgets, layouts, navigation, widget lifecycle
Code Quality	Readability, structure, naming, separation of concerns
API Handling	Async/await, error handling, loading states
State Management	Clean, scalable, proper pattern usage
Navigation & Routing	Screen transitions, data passing
Theming & Styling	Consistent ThemeData usage, reusable styles
Pagination	Smooth loading with proper states
UI/UX	Simple, intuitive, responsive design

Submission Guidelines

- GitHub repository link
- README with setup steps, architecture, screenshots

Bonus (Optional)

- Offline chat history (Hive / SharedPreferences)
- Dark mode support
- Animations in chat
- Unit or widget tests